Stage 3: Conversation Storage System - Architecture Analysis & Implementation Plan

Project: IGPT Enterprise Agentic AI Platform
Phase: Stage 3 - Production Storage & Database Architecture
Date: March 24, 2026
Scale Target: 100,000 daily active users

Executive Summary

This document provides a comprehensive analysis of the conversation storage system for IGPT, transitioning from an in-memory proof-of-concept (Stages 1-2) to a production-grade, scalable data architecture capable of handling 100,000 daily active users generating 10 million messages per day.

Current State: Authentication-enabled web application with multi-LLM support, using in-memory mock data.
Target State: Production-ready system with PostgreSQL (sharded), Redis caching, S3 artifact storage, vector search, and real-time streaming architecture.

Table of Contents

1. Current Implementation Status
2. Gap Analysis by Architecture Layer
3. Storage Strategy Deep Dive
4. Implementation Roadmap
5. Technical Specifications

6. Cost Analysis
7. Risk Assessment
8. Next Steps & Action Plan

---

1. Current Implementation Status

1.1 What's Completed (Stages 1-2)

✅ **Interface Layer (70% Complete)**

Implemented:

- Web interface with Next.js 15 + React 19
- REST API endpoints for conversations and messages
- Server-Sent Events (SSE) for LLM streaming
- Real-time message streaming to clients
- File upload support (images, PDFs, DOCX, code files)

Capabilities:

- WebSocket-like streaming via SSE
- REST: Standard HTTP methods (GET, POST, DELETE)
- File processing: PDF text extraction (unpdf), DOCX parsing (mammoth)

Missing:

- ❌ GraphQL endpoint for efficient data fetching
- ❌ WebSocket implementation (currently SSE only)

- ❌ Proper connection pooling and backpressure handling

✅ Gateway Layer (40% Complete)

Implemented:

- AWS Cognito User Pool authentication
- Email and phone number login with OTP
- Session management via Amplify Auth
- Basic routing through Next.js API routes

Capabilities:

- Authentication: User Pool with MFA (SMS optional)
- Token validation via Hub listeners
- Protected route guards in chat layout

Missing:

- ❌ AWS API Gateway for centralized routing
- ❌ Rate limiting per user/API key
- ❌ Request throttling and burst protection
- ❌ API key management for programmatic access
- ❌ Load balancing across multiple backend instances
- ❌ Role-based access control (RBAC) system
- ❌ Security headers and input sanitization at gateway level

✅ Services Layer (50% Complete)

Implemented:

- **Message Service:** `useStreamingChat` hook handles message creation and streaming
- **Conversation Service:** Basic CRUD operations in `/api/conversations` routes
- **LLM Orchestrator:** Multi-provider support (Claude, Mistral, GPT-4, Gemini, Qwen)
- **Model Router:** Dynamic model selection via Zustand store

Capabilities:

- Message sequencing in memory
- Conversation metadata management (title, timestamps)
- Multi-turn context handling
- File attachment processing

Missing:

- ❌ **Prompt Manager:** No centralized service for prompt templates and optimization
- ❌ **Artifact Manager:** No versioning system for code/document artifacts
- ❌ **Context Window Control:** No intelligent truncation of conversation history
- ❌ **Message Branching:** No support for editing previous prompts and creating forks
- ❌ **Session State Management:** Active/archived/deleted status not tracked
- ❌ **Background Job Processing:** No async workers for heavy operations

---

❌ Storage Layer (0% Complete - CRITICAL GAP)

Current State:

- All data stored in-memory (Zustand state)
- Mock conversations in `mock-data.ts`
- No persistent database
- File attachments stored as Blob URLs (temporary)
- No data survives page refresh

Required Components:

1. **PostgreSQL Database:**
    - Conversations table
    - Messages table
    - Artifacts table
    - Attachments table
    - User profiles table
2. **Redis Cache:**
    - Hot conversation data (last 5 turns)
    - Active user sessions
    - Rate limit counters
3. **S3 Object Storage:**
    - Artifact versioning (code, documents)
    - File attachments (PDFs, images, DOCX)
    - CloudFront CDN for delivery
4. **Vector Database (Pinecone/Weaviate):**
    - Message embeddings for semantic search

    - "Search past conversations" feature
5. **Message Queue (Kafka/RabbitMQ):**
    - Async write operations
    - Event streaming for analytics
    - Background job orchestration

Impact: This is the most critical gap preventing production deployment.

✅ **Processing Layer (80% Complete)**

Implemented:

- LLM orchestration with provider abstraction
- Model-specific formatting (Claude, Mistral, OpenAI, Google)
- Streaming response handling
- Error handling and retries

Capabilities:

- Multi-provider fallback
- Token counting and cost tracking (manual)
- Provider-specific prompt formatting

Missing:

- ❌ Message queue for async LLM calls
- ❌ Job scheduler for batch processing
- ❌ Dead letter queue for failed requests

- ❌ Circuit breaker for provider failures

❌ Observability Layer (0% Complete)

Missing:

- ❌ Centralized logging (ELK stack)
- ❌ Metrics collection (Prometheus)
- ❌ Performance monitoring (DataDog/New Relic)
- ❌ Error tracking (Sentry)
- ❌ Audit logs for compliance
- ❌ User analytics pipeline
- ❌ Cost tracking dashboard

1.2 Overall Completion Status

Layer	Completion	Priority	Blocking Production?
Interface	70%	Medium	No
Gateway	40%	High	Yes (rate limiting)
Services	50%	High	Partially
Storage	0%	**CRITICAL**	**YES**
Processing	80%	Medium	No

Layer	Completion	Priority	Blocking Production?
Observability	0%	High	Yes (compliance)

Critical Path: Storage Layer implementation is the primary blocker for production readiness.

---

2. Gap Analysis by Architecture Layer

2.1 Interface Layer Gaps

Missing: GraphQL Endpoint

Purpose: Efficiently fetch exactly the data needed by the UI.

Use Cases:

- Load conversation list with title + last message + user avatar in one query
- Fetch conversation metadata without pulling full message history
- Real-time subscriptions for typing indicators and presence

Implementation Plan:

```typescript
```

```
// GraphQL schema example
type Query {
  conversation(id: ID!): Conversation
  conversations(userId: ID!, limit: Int): [Conversation!]!
  searchMessages(query: String!, userId: ID!): [Message!]!
}

type Subscription {
  messageAdded(conversationId: ID!): Message
  typingIndicator(conversationId: ID!): TypingEvent
}

type Conversation {
  id: ID!
  title: String!
  lastMessage: Message
  messageCount: Int!
  createdAt: DateTime!
  updatedAt: DateTime!
  user: User!
}
```

Technology Stack:

- Apollo Server (Node.js)
- GraphQL Subscriptions over WebSocket
- DataLoader for batching and caching

Effort: 2-3 weeks (Medium complexity)

Missing: True WebSocket Support

Current: SSE (Server-Sent Events) for streaming
Gap: No bidirectional communication

Why WebSockets Matter:

1. **Bi-directional:** Server can push updates without client polling
2. **Lower Latency:** Persistent connection eliminates HTTP overhead
3. **Typing Indicators:** Real-time user presence
4. **Collaborative Features:** Multi-user editing support

Implementation:

```typescript
// WebSocket server (Socket.IO)
io.on('connection', (socket) => {
  socket.on('join_conversation', (conversationId) => {
    socket.join(`conversation:${conversationId}`);
  });

  socket.on('typing_start', (conversationId) => {
    socket.to(`conversation:${conversationId}`).emit('user_typing', {
      userId: socket.userId,
      timestamp: Date.now()
    });
  });
});
```

Effort: 1-2 weeks (Low-Medium complexity)

2.2 Gateway Layer Gaps

Missing: AWS API Gateway Integration

Current: Direct Next.js API routes
Gap: No centralized traffic management

Benefits of API Gateway:

1. **Rate Limiting:** Throttle requests per user/API key
    - Free tier: 1000 requests/hour
    - Pro tier: 10,000 requests/hour
    - Enterprise: Unlimited
2. **Request Validation:** JSON schema validation before hitting backend
3. **Usage Tracking:** Monitor API consumption per client
4. **Caching:** Cache GET responses at edge locations
5. **Multi-Region:** Deploy endpoints globally

Implementation:

```yaml
```

```yaml
# AWS API Gateway REST API
Resources:
  IGPTApi:
    Type: AWS::ApiGateway::RestApi
    Properties:
      Name: igpt-production-api

  ConversationsResource:
    Type: AWS::ApiGateway::Resource
    Properties:
      RestApiId: !Ref IGPTApi
      ParentId: !GetAtt IGPTApi.RootResourceId
      PathPart: conversations

  UsagePlan:
    Type: AWS::ApiGateway::UsagePlan
    Properties:
      Quota:
        Limit: 10000
        Period: MONTH
      Throttle:
        BurstLimit: 200
        RateLimit: 100
```

Effort: 2-3 weeks (Medium complexity)

Missing: Role-Based Access Control (RBAC)

Current: Binary authentication (logged in or not)
Gap: No permission system

Required Roles:

```typescript
enum Role {
  ADMIN = 'admin',            // Full system access
  MANAGER = 'manager',        // Team management + reporting
  DEVELOPER = 'developer',    // API access + Claude Code integration
  USER = 'user',              // Standard chat access
  VIEWER = 'viewer'           // Read-only access
}

interface Permission {
  resource: 'conversation' | 'artifact' | 'model' | 'settings';
  action: 'create' | 'read' | 'update' | 'delete';
  scope: 'own' | 'team' | 'all';
}

// Example: Manager can view all team conversations
{
  role: 'manager',
  permissions: [
    { resource: 'conversation', action: 'read', scope: 'team' },
    { resource: 'artifact', action: 'read', scope: 'team' }
  ]
}
```

Database Schema:

```sql
```

```sql
CREATE TABLE roles (
  id UUID PRIMARY KEY,
  name VARCHAR(50) UNIQUE NOT NULL,
  description TEXT
);

CREATE TABLE permissions (
  id UUID PRIMARY KEY,
  role_id UUID REFERENCES roles(id),
  resource VARCHAR(50) NOT NULL,
  action VARCHAR(50) NOT NULL,
  scope VARCHAR(50) NOT NULL
);

CREATE TABLE user_roles (
  user_id UUID REFERENCES users(id),
  role_id UUID REFERENCES roles(id),
  workspace_id UUID,
  granted_at TIMESTAMP DEFAULT NOW(),
  PRIMARY KEY (user_id, role_id, workspace_id)
);
```

Effort: 3-4 weeks (High complexity)

2.3 Services Layer Gaps

Missing: Prompt Manager Service

Purpose: Centralized management of system prompts, user preferences, and templates.

Features Needed:

1. **Prompt Templates:** Pre-built prompts for common tasks
2. **Style Configurations:** Already in Zustand, needs persistence
3. **Prompt Optimization:** Track successful prompts, A/B testing
4. **Version Control:** Track changes to system prompts over time

Database Schema:

```sql
CREATE TABLE prompt_templates (
  id UUID PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  category VARCHAR(50), -- 'HR', 'Technical', 'Business'
  prompt_text TEXT NOT NULL,
  variables JSONB,      -- { "name": "string", "company": "string" }
  usage_count INT DEFAULT 0,
  success_rate DECIMAL(5,2),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE prompt_history (
  id UUID PRIMARY KEY,
  template_id UUID REFERENCES prompt_templates(id),
  version INT NOT NULL,
  prompt_text TEXT NOT NULL,
  changed_by UUID REFERENCES users(id),
  changed_at TIMESTAMP DEFAULT NOW()
);
```

API Design:

```typescript
// Prompt Manager Service
class PromptManagerService {
  async createTemplate(template: PromptTemplate): Promise<PromptTemplate>;
  async getTemplate(id: string): Promise<PromptTemplate>;
  async listTemplates(category?: string): Promise<PromptTemplate[]>;
  async updateTemplate(id: string, updates: Partial<PromptTemplate>): Promise<P
  async renderTemplate(id: string, variables: Record<string, any>): Promise<str
  async trackUsage(id: string, success: boolean): Promise<void>;
}
```

Effort: 2-3 weeks (Medium complexity)

Missing: Artifact Manager with Versioning

Current: Artifacts stored in Zustand, no versioning
Gap: No history, no rollback, no S3 persistence

Requirements:

1. **Version Control:** Track all changes to code/document artifacts
2. **S3 Storage:** Persist artifacts with CDN delivery
3. **Diff Tracking:** Show changes between versions
4. **Rollback:** Restore previous versions
5. **Metadata:** Language, size, timestamps, author

Database Schema:

```sql
```

```sql
CREATE TABLE artifacts (
  id UUID PRIMARY KEY,
  conversation_id UUID REFERENCES conversations(id),
  artifact_type VARCHAR(50) NOT NULL, -- 'code', 'document', 'html', 'react'
  title VARCHAR(255),
  language VARCHAR(50),
  current_version INT DEFAULT 1,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE artifact_versions (
  id UUID PRIMARY KEY,
  artifact_id UUID REFERENCES artifacts(id),
  version INT NOT NULL,
  content TEXT NOT NULL,
  size_bytes INT,
  storage_path VARCHAR(500), -- S3: s3://bucket/artifacts/{id}/v{version}.txt
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(artifact_id, version)
);

CREATE INDEX idx_artifacts_conversation ON artifacts(conversation_id);
CREATE INDEX idx_artifact_versions_artifact ON artifact_versions(artifact_id);
```

S3 Structure:

```
s3://igpt-artifacts-prod/
  ├── artifacts/
  |   ├── {workspace_id}/
  |   |   ├── {conversation_id}/
  |   |   |   ├── {artifact_id}/
  |   |   |   |   ├── v1.py
  |   |   |   |   ├── v2.py
  |   |   |   |   ├── v3.py
  |   |   |   |   └── metadata.json
```

API Design:

```typescript
class ArtifactManagerService {
  async createArtifact(artifact: ArtifactInput): Promise<Artifact>;
  async updateArtifact(id: string, content: string): Promise<ArtifactVersion>;
  async getArtifact(id: string, version?: number): Promise<ArtifactVersion>;
  async listVersions(artifactId: string): Promise<ArtifactVersion[]>;
  async downloadArtifact(id: string, version: number): Promise<string>; // S3 s
  async diffVersions(id: string, v1: number, v2: number): Promise<Diff>;
  async rollback(id: string, toVersion: number): Promise<ArtifactVersion>;
}
```

Effort: 3-4 weeks (High complexity)

2.4 Storage Layer Gaps (CRITICAL)

This is the **highest priority** gap preventing production deployment.

Missing: PostgreSQL Database

Current: In-memory mock data
Target: AWS Aurora PostgreSQL with sharding

Complete Schema Design:

```sql
```

```sql
-- ==============================================
-- Users & Authentication
-- ==============================================
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  cognito_sub VARCHAR(255) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE,
  phone_number VARCHAR(50) UNIQUE,
  name VARCHAR(255),
  avatar_url VARCHAR(500),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  last_active_at TIMESTAMP
);

CREATE INDEX idx_users_cognito_sub ON users(cognito_sub);
CREATE INDEX idx_users_email ON users(email);

-- ==============================================
-- Workspaces (for multi-tenancy)
-- ==============================================
CREATE TABLE workspaces (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  owner_id UUID REFERENCES users(id),
  plan VARCHAR(50) DEFAULT 'free', -- 'free', 'pro', 'enterprise'
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE workspace_members (
  workspace_id UUID REFERENCES workspaces(id),
```

```sql
  user_id UUID REFERENCES users(id),
  role VARCHAR(50) DEFAULT 'member',
  joined_at TIMESTAMP DEFAULT NOW(),
  PRIMARY KEY (workspace_id, user_id)
);

-- ============================================
-- Conversations
-- ============================================
CREATE TABLE conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  workspace_id UUID REFERENCES workspaces(id),
  title VARCHAR(500) NOT NULL DEFAULT 'New Chat',
  message_count INT DEFAULT 0,
  status VARCHAR(50) DEFAULT 'active', -- 'active', 'archived', 'deleted'
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  metadata JSONB DEFAULT '{}'::jsonb
);

-- Sharding key: user_id
-- All queries MUST include WHERE user_id = ? to avoid cross-shard joins
CREATE INDEX idx_conversations_user_id ON conversations(user_id);
CREATE INDEX idx_conversations_workspace_id ON conversations(workspace_id);
CREATE INDEX idx_conversations_updated_at ON conversations(updated_at DESC);
CREATE INDEX idx_conversations_status ON conversations(status) WHERE status !=

-- ============================================
-- Messages
-- ============================================
CREATE TABLE messages (
```

```sql
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id), -- Denormalized for sharding
  role VARCHAR(20) NOT NULL CHECK (role IN ('user', 'assistant', 'system')),
  content TEXT NOT NULL,
  tokens_used INT,
  model VARCHAR(255),
  provider VARCHAR(50),
  thinking_steps JSONB,
  created_at TIMESTAMP DEFAULT NOW(),
  metadata JSONB DEFAULT '{}'::jsonb
);

-- Sharding key: user_id (denormalized from conversation)
CREATE INDEX idx_messages_conversation_id ON messages(conversation_id);
CREATE INDEX idx_messages_user_id ON messages(user_id);
CREATE INDEX idx_messages_created_at ON messages(created_at DESC);

-- ============================================
-- Artifacts
-- ============================================
CREATE TABLE artifacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id), -- Denormalized for sharding
  artifact_type VARCHAR(50) NOT NULL, -- 'code', 'document', 'html', 'react'
  title VARCHAR(500),
  language VARCHAR(50),
  current_version INT DEFAULT 1,
  storage_path VARCHAR(500), -- S3 base path
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
```

```sql
);

CREATE TABLE artifact_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  artifact_id UUID NOT NULL REFERENCES artifacts(id) ON DELETE CASCADE,
  version INT NOT NULL,
  content TEXT, -- Small artifacts stored inline
  size_bytes INT,
  storage_path VARCHAR(500), -- S3: s3://bucket/artifacts/{id}/v{version}
  created_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(artifact_id, version)
);

CREATE INDEX idx_artifacts_conversation_id ON artifacts(conversation_id);
CREATE INDEX idx_artifacts_user_id ON artifacts(user_id);
CREATE INDEX idx_artifact_versions_artifact ON artifact_versions(artifact_id);

-- ============================================
-- Attachments
-- ============================================
CREATE TABLE attachments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  message_id UUID NOT NULL REFERENCES messages(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id), -- Denormalized for sharding
  file_name VARCHAR(500) NOT NULL,
  file_type VARCHAR(100),
  size_bytes INT,
  storage_path VARCHAR(500) NOT NULL, -- S3: s3://bucket/attachments/{user_id}/
  extracted_content TEXT, -- PDF/DOCX text extraction
  uploaded_at TIMESTAMP DEFAULT NOW()
);
```

```sql
CREATE INDEX idx_attachments_message_id ON attachments(message_id);
CREATE INDEX idx_attachments_user_id ON attachments(user_id);

-- ==============================================
-- Usage Tracking
-- ==============================================
CREATE TABLE usage_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  conversation_id UUID REFERENCES conversations(id),
  event_type VARCHAR(50) NOT NULL, -- 'message_sent', 'artifact_created', 'sear
  tokens_used INT,
  cost_usd DECIMAL(10, 6),
  model VARCHAR(255),
  provider VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_usage_logs_user_id ON usage_logs(user_id);
CREATE INDEX idx_usage_logs_created_at ON usage_logs(created_at DESC);

-- ==============================================
-- Audit Logs (Compliance)
-- ==============================================
CREATE TABLE audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  action VARCHAR(100) NOT NULL, -- 'conversation.created', 'message.deleted', '
  resource_type VARCHAR(50),
  resource_id UUID,
  ip_address VARCHAR(50),
  user_agent TEXT,
```

```sql
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Separate table space for compliance
CREATE INDEX idx_audit_logs_user_id ON audit_logs(user_id);
CREATE INDEX idx_audit_logs_created_at ON audit_logs(created_at DESC);
CREATE INDEX idx_audit_logs_action ON audit_logs(action);
```

Migration Strategy:

1. **Phase 1:** Set up AWS Aurora PostgreSQL cluster
2. **Phase 2:** Deploy schema to development environment
3. **Phase 3:** Write database access layer (Prisma ORM)
4. **Phase 4:** Migrate mock data to PostgreSQL
5. **Phase 5:** Update API routes to use database
6. **Phase 6:** Test with production load (stress testing)

Effort: 4-6 weeks (Very High complexity)

Missing: Redis Cache Layer

Purpose: Reduce database load, improve response times

Cache Strategy:

```typescript
```

```typescript
// Cache key patterns
const cacheKeys = {
  // Hot conversation data (1 hour TTL)
  conversationLatest: (convId: string) => `conversation:${convId}:latest`,

  // User's active conversations (30 min TTL)
  userConversations: (userId: string) => `user:${userId}:conversations:active`,

  // Artifact metadata (24 hour TTL)
  artifacts: (convId: string) => `conversation:${convId}:artifacts`,

  // Rate limiting counters (1 hour window)
  rateLimit: (userId: string) => `rate_limit:${userId}`,

  // User session data (until logout)
  session: (sessionId: string) => `session:${sessionId}`
};
```

Implementation:

```typescript
```

```
import { Redis } from 'ioredis';

const redis = new Redis({
  host: process.env.REDIS_HOST,
  port: 6379,
  password: process.env.REDIS_PASSWORD,
  retryStrategy: (times) => Math.min(times * 50, 2000)
});

class ConversationCache {
  // Get last 5 messages from cache
  async getLatestMessages(conversationId: string): Promise<Message[] | null> {
    const cached = await redis.get(cacheKeys.conversationLatest(conversationId)
    return cached ? JSON.parse(cached) : null;
  }

  // Cache last 5 messages (1 hour TTL)
  async cacheLatestMessages(conversationId: string, messages: Message[]): Promi
    await redis.setex(
      cacheKeys.conversationLatest(conversationId),
      3600, // 1 hour
      JSON.stringify(messages.slice(-5))
    );
  }

  // Invalidate on new message
  async invalidate(conversationId: string): Promise<void> {
    await redis.del(cacheKeys.conversationLatest(conversationId));
  }
}
```

Cache Invalidation Strategy:

- **Write-through:** Write to DB first, then update cache
- **Lazy Loading:** On cache miss, fetch from DB and populate cache
- **TTL-based Expiration:** Automatic cleanup of stale data

AWS ElastiCache Configuration:

```yaml
Redis Cluster:
  Node Type: cache.r6g.large (13.07 GB memory)
  Nodes: 3 replicas (high availability)
  Multi-AZ: Enabled
  Encryption: In-transit + at-rest
  Cost: ~$450/month
```

Effort: 2-3 weeks (Medium complexity)

Missing: S3 Object Storage for Artifacts

Current: Blob URLs (temporary, browser-only)
Target: S3 with CloudFront CDN

Directory Structure:

```
s3://igpt-artifacts-prod/
  ├── artifacts/
  |   ├── {workspace_id}/
  |   |   ├── {conversation_id}/
  |   |   |   ├── {artifact_id}/
  |   |   |   |   ├── v1.py          # Version 1
  |   |   |   |   ├── v2.py          # Version 2
  |   |   |   |   ├── v3.py          # Version 3 (current)
  |   |   |   |   └── metadata.json   # Artifact metadata
  |
  ├── attachments/
  |   ├── {user_id}/
  |   |   ├── {conversation_id}/
  |   |   |   ├── uploaded_file.pdf
  |   |   |   ├── screenshot.png
  |   |   |   └── document.docx
```

S3 Upload Service:

```typescript
```

```
import { S3Client, PutObjectCommand, GetObjectCommand } from '@aws-sdk/client-s
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

class S3StorageService {
  private s3Client: S3Client;
  private bucketName = 'igpt-artifacts-prod';

  constructor() {
    this.s3Client = new S3Client({
      region: 'us-east-1',
      credentials: {
        accessKeyId: process.env.AWS_ACCESS_KEY_ID!,
        secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY!
      }
    });
  }

  // Upload artifact version
  async uploadArtifact(
    artifactId: string,
    version: number,
    content: string,
    language: string
  ): Promise<string> {
    const key = `artifacts/${artifactId}/v${version}.${this.getExtension(langua

    await this.s3Client.send(new PutObjectCommand({
      Bucket: this.bucketName,
      Key: key,
      Body: content,
      ContentType: this.getContentType(language),
      Metadata: {
```

```
      artifactId,
      version: version.toString(),
      language
    }
  }));

  return `s3://${this.bucketName}/${key}`;
}

// Generate signed URL for download (expires in 1 hour)
async getDownloadUrl(storagePath: string): Promise<string> {
  const key = storagePath.replace(`s3://${this.bucketName}/`, '');

  const command = new GetObjectCommand({
    Bucket: this.bucketName,
    Key: key
  });

  return await getSignedUrl(this.s3Client, command, { expiresIn: 3600 });
}

private getExtension(language: string): string {
  const extensions: Record<string, string> = {
    'python': 'py',
    'javascript': 'js',
    'typescript': 'ts',
    'java': 'java',
    'html': 'html',
    'css': 'css'
  };
  return extensions[language] || 'txt';
}
```

```
  private getContentType(language: string): string {
    const types: Record<string, string> = {
      'python': 'text/x-python',
      'javascript': 'application/javascript',
      'typescript': 'application/typescript',
      'html': 'text/html',
      'css': 'text/css'
    };
    return types[language] || 'text/plain';
  }
}
```

CloudFront CDN Configuration:

```yaml
CloudFront Distribution:
  Origin: s3://igpt-artifacts-prod
  Cache Behavior:
    - Path: /artifacts/*
      TTL: 86400 (24 hours)
      Compress: true
    - Path: /attachments/*
      TTL: 3600 (1 hour)
      Compress: true
  SSL Certificate: ACM certificate
  Geo Restrictions: None
  Price Class: All edge locations
```

Benefits:

- **Performance:** Global CDN with <50ms latency
- **Cost:** ~$0.085/GB transfer (vs $0.09/GB from S3)
- **Reliability:** 99.99% SLA
- **Security:** Signed URLs prevent unauthorized access

Effort: 2-3 weeks (Medium complexity)

Missing: Vector Database for Semantic Search

Purpose: Enable "search past conversations" feature with natural language

Architecture:

```
User query: "What did we discuss about machine learning?"
    ↓
Embedding Model: text-embedding-ada-002 (OpenAI)
    ↓
Vector: [0.123, -0.456, 0.789, ..., 0.234] (1536 dimensions)
    ↓
Vector DB Query: Find top 10 similar messages
    ↓
Results: Messages sorted by similarity score
```

Implementation:

```typescript
```

```typescript
import { Pinecone } from '@pinecone-database/pinecone';
import OpenAI from 'openai';

class VectorSearchService {
  private pinecone: Pinecone;
  private openai: OpenAI;
  private indexName = 'igpt-messages';

  constructor() {
    this.pinecone = new Pinecone({
      apiKey: process.env.PINECONE_API_KEY!,
      environment: 'us-east-1-aws'
    });

    this.openai = new OpenAI({
      apiKey: process.env.OPENAI_API_KEY!
    });
  }

  // Generate embedding for message
  async embedMessage(content: string): Promise<number[]> {
    const response = await this.openai.embeddings.create({
      model: 'text-embedding-ada-002',
      input: content
    });

    return response.data[0].embedding;
  }

  // Index new message
  async indexMessage(message: Message): Promise<void> {
    const embedding = await this.embedMessage(message.content);
```

```
  const index = this.pinecone.index(this.indexName);
  await index.upsert([{
    id: message.id,
    values: embedding,
    metadata: {
      conversationId: message.conversationId,
      userId: message.user_id,
      role: message.role,
      content: message.content.substring(0, 500), // Store snippet
      createdAt: message.created_at.toISOString()
    }
  }]);
}

// Search messages by natural language
async searchMessages(
  query: string,
  userId: string,
  limit: number = 10
): Promise<SearchResult[]> {
  const queryEmbedding = await this.embedMessage(query);

  const index = this.pinecone.index(this.indexName);
  const results = await index.query({
    vector: queryEmbedding,
    topK: limit,
    filter: { userId: { $eq: userId } }, // Only search user's messages
    includeMetadata: true
  });

  return results.matches.map(match => ({
```

```
      messageId: match.id,
      conversationId: match.metadata.conversationId as string,
      content: match.metadata.content as string,
      score: match.score,
      createdAt: new Date(match.metadata.createdAt as string)
    }));
  }
}
```

Database Schema Extension:

```sql
-- Track embedding status
ALTER TABLE messages ADD COLUMN embedding_status VARCHAR(20) DEFAULT 'pending';
ALTER TABLE messages ADD COLUMN embedding_indexed_at TIMESTAMP;

CREATE INDEX idx_messages_embedding_status ON messages(embedding_status)
  WHERE embedding_status = 'pending';
```

Background Job (Celery/Bull):

```typescript
```

```
// Process messages in batches
async function processEmbeddingQueue() {
  const pendingMessages = await db.query(`
    SELECT * FROM messages
    WHERE embedding_status = 'pending'
    LIMIT 100
  `);

  for (const message of pendingMessages) {
    try {
      await vectorSearch.indexMessage(message);

      await db.query(`
        UPDATE messages
        SET embedding_status = 'indexed',
            embedding_indexed_at = NOW()
        WHERE id = $1
      `, [message.id]);
    } catch (error) {
      console.error(`Failed to index message ${message.id}:`, error);

      await db.query(`
        UPDATE messages
        SET embedding_status = 'failed'
        WHERE id = $1
      `, [message.id]);
    }
  }
}
```

```
// Run every 5 minutes
setInterval(processEmbeddingQueue, 5 * 60 * 1000);
```

Pinecone Configuration:

```yaml
Index: igpt-messages
Dimensions: 1536 (OpenAI ada-002)
Metric: cosine similarity
Pods: 1x s1.x1 (1M vectors capacity)
Replicas: 3 (high availability)
Cost: ~$70/month
```

Effort: 3-4 weeks (High complexity)

Missing: Message Queue (Kafka/RabbitMQ)

Purpose: Decouple request handling from heavy I/O operations

Use Cases:

1. **Async Database Writes:** User gets instant response, write happens in background
2. **LLM Request Queue:** Buffer LLM requests, handle provider failures
3. **Event Streaming:** Feed analytics pipeline with user actions
4. **Background Jobs:** Embedding generation, artifact processing, email notifications

Architecture:

```
Client Request
    ↓
API Endpoint (Next.js)
    ↓
Publish to Kafka → Return 202 Accepted to client (instant)
    ↓
Worker Process (Node.js consumer)
    ↓
[PostgreSQL Write] + [Redis Cache Update] + [Vector Index]
```

Kafka Implementation:

```typescript
```

```typescript
import { Kafka, Producer, Consumer } from 'kafkajs';

class MessageQueueService {
  private kafka: Kafka;
  private producer: Producer;

  constructor() {
    this.kafka = new Kafka({
      clientId: 'igpt-producer',
      brokers: ['kafka-1.igpt.internal:9092', 'kafka-2.igpt.internal:9092'],
      ssl: true,
      sasl: {
        mechanism: 'scram-sha-256',
        username: process.env.KAFKA_USERNAME!,
        password: process.env.KAFKA_PASSWORD!
      }
    });

    this.producer = this.kafka.producer();
  }

  // Publish new message event
  async publishMessage(message: Message): Promise<void> {
    await this.producer.send({
      topic: 'messages.created',
      messages: [{
        key: message.conversationId,
        value: JSON.stringify(message),
        partition: this.getPartition(message.user_id) // Partition by user for
      }]
    });
  }
```

```typescript
// Consumer (worker process)
async consumeMessages() {
  const consumer = this.kafka.consumer({ groupId: 'message-writers' });
  await consumer.connect();
  await consumer.subscribe({ topic: 'messages.created', fromBeginning: false

  await consumer.run({
    eachMessage: async ({ topic, partition, message }) => {
      const msg = JSON.parse(message.value!.toString()) as Message;

      try {
        // 1. Write to PostgreSQL
        await db.insertMessage(msg);

        // 2. Invalidate cache
        await redis.del(cacheKeys.conversationLatest(msg.conversationId));

        // 3. Queue embedding job
        await embeddingQueue.add('index-message', { messageId: msg.id });

        console.log(`✅ Processed message ${msg.id}`);
      } catch (error) {
        console.error(`❌ Failed to process message ${msg.id}:`, error);
        // Dead letter queue handling
        await this.sendToDeadLetterQueue(msg, error);
      }
    }
  });
}

private getPartition(userId: string): number {
```

```
    // Hash user ID to partition (ensures ordering per user)
    const hash = userId.split('').reduce((acc, char) => acc + char.charCodeAt(0
    return hash % 10; // 10 partitions
  }

  private async sendToDeadLetterQueue(message: Message, error: unknown) {
    await this.producer.send({
      topic: 'messages.dlq',
      messages: [{
        key: message.id,
        value: JSON.stringify({ message, error: String(error) })
      }]
    });
  }
}
```

Benefits:

1. **Performance:** Sub-1ms response to client (vs 5-10ms with sync DB write)
2. **Reliability:** Kafka guarantees at-least-once delivery
3. **Scalability:** Horizontal scaling of worker processes
4. **Decoupling:** Services don't directly depend on each other

AWS MSK (Managed Kafka) Configuration:

```yaml
```

```
Cluster: igpt-kafka-prod
Brokers: 3 nodes (kafka.m5.large)
Storage: 1TB per broker (auto-scaling)
Partitions: 10 per topic
Replication Factor: 3
Retention: 7 days
Cost: ~$1,500/month
```

Effort: 4-5 weeks (Very High complexity)

---

3. Storage Strategy Deep Dive

3.1 Database Selection Rationale

Why PostgreSQL over MongoDB at 100k user scale?

Criterion	PostgreSQL (Chosen)	MongoDB
ACID Transactions	✅ Full support	⚠️ Multi-document transactions complex
Query Flexibility	✅ Rich SQL, CTEs, window functions	⚠️ Aggregation pipeline learning curve
Atomic Updates	✅ `UPDATE conversations SET message_count = message_count + 1`	❌ Requires separate increment command
Cost (100k users)	✅ $15k/month (Aurora)	❌ $25k+/month (Atlas sharded)
Maturity	✅ 30+ years, battle-tested	⚠️ Younger, some gotchas
JSON Support	✅ JSONB with indexing	✅ Native
Scaling	✅ Sharding with Citus/Aurora	✅ Native sharding
Managed Service	✅ AWS Aurora PostgreSQL	✅ MongoDB Atlas

Decision: PostgreSQL with JSONB columns for flexible metadata provides the best balance of reliability, cost, and developer experience.

3.2 Sharding Strategy

Shard Key: `user_id`
Why?

1. **Stable:** User ID never changes
2. **Known at Request Time:** Every API call includes authentication
3. **Naturally Partitions Data:** Users don't share conversations
4. **Avoids Hotspots:** Even distribution assuming users are evenly distributed

Shard Configuration:

```
8 PostgreSQL shards × 3 replicas = 24 database instances
Capacity per shard: 12,500 users
Total capacity: 100,000 users
```

Shard Routing Logic:

```typescript
```

```
function getShardId(userId: string): number {
  // Use consistent hashing to map user to shard
  const hash = createHash('sha256').update(userId).digest('hex');
  const numericHash = parseInt(hash.substring(0, 8), 16);
  return numericHash % 8; // 8 shards
}

// Example: Route query to correct shard
async function getConversations(userId: string) {
  const shardId = getShardId(userId);
  const db = databasePool.getShard(shardId);

  return await db.query(`
    SELECT * FROM conversations
    WHERE user_id = $1
    ORDER BY updated_at DESC
    LIMIT 20
  `, [userId]);
}
```

Important: All queries MUST include `user_id` in the `WHERE` clause to avoid expensive cross-shard joins.

3.3 Caching Strategy

Redis Cache Layers:

1. **Hot Conversation Cache (TTL: 1 hour)**
   - Key: `conversation:{id}:latest`
   - Value: Last 5 messages (JSON)

    - Invalidated on new message
2. **User Active Conversations (TTL: 30 minutes)**
    - Key: `user:{userId}:conversations:active`
    - Value: List of conversation IDs + titles
    - Invalidated on conversation create/update
3. **Artifact Metadata (TTL: 24 hours)**
    - Key: `conversation:{id}:artifacts`
    - Value: List of artifact references
    - Rarely changes
4. **Rate Limiting Counters (TTL: 1 hour)**
    - Key: `rate_limit:{userId}:hour`
    - Value: Request count (incremented on each API call)
    - Used by API Gateway

Cache Hit Ratio Target: 80%+ for conversation reads

3.4 Streaming Architecture

Current: SSE (Server-Sent Events)
Enhancement Needed: Add buffering layer

Problem with Current Architecture:

```
Client Request
    ↓
Next.js API Route
    ↓
LLM Provider (Claude/OpenAI) ← Direct connection, no buffering
    ↓
Stream tokens directly to client
```

If LLM responds faster than client can consume:

- Backpressure builds up
- Memory usage spikes
- Server crashes under load

Production Architecture:

```
Client Request (SSE connection)
    ↓
Next.js API Route
    ↓
Kafka Topic: llm.requests
    ↓
Worker Process (consumes from Kafka)
    ↓
LLM Provider
    ↓
Kafka Topic: llm.responses.{conversationId}
    ↓
SSE Server (reads from Kafka, streams to client)
    ↓
Redis Pub/Sub (fan-out to multiple clients)
    ↓
Client receives tokens in real-time
```

Benefits:

1. **Buffering:** Kafka handles burst traffic
2. **Reliability:** If client disconnects, can resume from offset
3. **Scalability:** Multiple workers process LLM requests in parallel
4. **Observability:** Every token logged for debugging

Database Write Strategy:

- Batch writes every 5 tokens OR 100ms (whichever comes first)
- Reduces DB IOPS from 1000 writes/sec to 200 writes/sec

3.5 Backup & Retention

PostgreSQL Backup:

- **Full Backup:** Daily at 3 AM UTC
- **Incremental Backup:** Every 6 hours
- **WAL Archiving:** Continuous (point-in-time recovery)
- **Retention:** 30 days

S3 Backup:

- **Versioning:** Enabled (immutable objects)
- **Glacier Transition:** After 90 days (99% cost reduction)
- **Deletion Protection:** Multi-factor authentication required

Compliance:

```sql
```

```sql
-- Audit log table (separate tablespace)
CREATE TABLE audit_logs (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  action VARCHAR(100) NOT NULL,
  resource_type VARCHAR(50),
  resource_id UUID,
  ip_address VARCHAR(50),
  user_agent TEXT,
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW()
) PARTITION BY RANGE (created_at);

-- Monthly partitions for efficient archival
CREATE TABLE audit_logs_2026_03 PARTITION OF audit_logs
  FOR VALUES FROM ('2026-03-01') TO ('2026-04-01');
```

Retention Policy:

- **Messages:** Keep forever (or 7 years for compliance)
- **Attachments:** Keep 1 year, then delete or move to Glacier
- **Audit Logs:** Keep 7 years (regulatory requirement)
- **Usage Logs:** Aggregate to monthly summaries after 90 days

---

4. Implementation Roadmap

Phase 1: Database Foundation (Weeks 1-6)

Week 1-2: PostgreSQL Setup

Tasks:

- [ ] Provision AWS Aurora PostgreSQL cluster (3 nodes)
- [ ] Deploy database schema
- [ ] Configure automated backups and monitoring
- [ ] Set up connection pooling (PgBouncer)
- [ ] Create read replica for analytics queries

Deliverables:

- Aurora cluster running in `us-east-1`
- Schema deployed with indexes
- Backup strategy tested
- Connection pool configured (max 100 connections per node)

Owner: Database Engineer
Dependencies: AWS account with Aurora permissions

Week 3-4: Database Access Layer

Tasks:

- [ ] Install Prisma ORM
- [ ] Generate Prisma client from schema
- [ ] Write repository layer (ConversationRepository, MessageRepository, etc.)
- [ ] Add connection health checks
- [ ] Write integration tests

Code Structure:

```typescript
```

```
// src/lib/db/prisma.ts
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient({
  log: ['query', 'info', 'warn', 'error'],
  datasources: {
    db: {
      url: process.env.DATABASE_URL
    }
  }
});

export default prisma;

// src/lib/db/repositories/conversation.repository.ts
import prisma from '../prisma';
import { Conversation, Message } from '@/types';

export class ConversationRepository {
  async create(userId: string, title: string): Promise<Conversation> {
    return await prisma.conversation.create({
      data: {
        userId,
        title,
        status: 'active'
      }
    });
  }

  async findByUserId(userId: string, limit: number = 20): Promise<Conversation[
    return await prisma.conversation.findMany({
      where: { userId, status: 'active' },
```

```
      orderBy: { updatedAt: 'desc' },
      take: limit,
      include: {
        _count: {
          select: { messages: true }
        }
      }
    });
  }

  async addMessage(conversationId: string, message: Omit<Message, 'id'>): Promi
    // Transactional: create message + increment count
    return await prisma.$transaction(async (tx) => {
      const msg = await tx.message.create({
        data: {
          conversationId,
          ...message
        }
      });

      await tx.conversation.update({
        where: { id: conversationId },
        data: {
          messageCount: { increment: 1 },
          updatedAt: new Date()
        }
      });

      return msg;
    });
  }
}
```

Deliverables:

- Prisma schema matching SQL schema
- Repository pattern implemented for all entities
- 80%+ test coverage
- Database migrations working

Owner: Backend Developer
Dependencies: PostgreSQL cluster operational

Week 5-6: API Migration

Tasks:

- [ ] Update `/api/conversations` routes to use PostgreSQL
- [ ] Update `/api/conversations/[id]/messages` to use database
- [ ] Migrate mock data to database (seed script)
- [ ] Add error handling and retry logic
- [ ] Performance testing (1000 req/sec load test)

Before (Mock Data):

```typescript
// src/app/api/conversations/route.ts
export async function GET() {
  const conversations = mockConversations; // In-memory
  return NextResponse.json(conversations);
}
```

After (PostgreSQL):

```typescript
import { ConversationRepository } from '@/lib/db/repositories/conversation.repo

export async function GET(request: Request) {
  try {
    const { searchParams } = new URL(request.url);
    const userId = searchParams.get('userId');

    if (!userId) {
      return NextResponse.json({ error: 'userId required' }, { status: 400 });
    }

    const repo = new ConversationRepository();
    const conversations = await repo.findByUserId(userId);

    return NextResponse.json(conversations);
  } catch (error) {
    console.error('Failed to fetch conversations:', error);
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
  }
}
```

Deliverables:

- All API routes using PostgreSQL
- Mock data removed from codebase

- Load testing results documented
- Error handling standardized

Owner: Full-Stack Developer
Dependencies: Database access layer complete

Phase 2: Caching & Performance (Weeks 7-10)

Week 7-8: Redis Integration

Tasks:

- [ ] Provision AWS ElastiCache for Redis
- [ ] Implement Redis client wrapper
- [ ] Add caching layer to repositories
- [ ] Implement cache invalidation logic
- [ ] Monitor cache hit ratio

Code Example:

```typescript
```

```typescript
// src/lib/cache/redis.ts
import Redis from 'ioredis';

const redis = new Redis({
  host: process.env.REDIS_HOST,
  port: 6379,
  password: process.env.REDIS_PASSWORD,
  retryStrategy: (times) => Math.min(times * 50, 2000),
  maxRetriesPerRequest: 3
});

export class CacheService {
  async get<T>(key: string): Promise<T | null> {
    try {
      const cached = await redis.get(key);
      return cached ? JSON.parse(cached) : null;
    } catch (error) {
      console.error(`Cache GET error for key ${key}:`, error);
      return null; // Fail gracefully
    }
  }

  async set(key: string, value: unknown, ttl: number = 3600): Promise<void> {
    try {
      await redis.setex(key, ttl, JSON.stringify(value));
    } catch (error) {
      console.error(`Cache SET error for key ${key}:`, error);
      // Don't throw - cache is not critical path
    }
  }

  async del(key: string): Promise<void> {
```

```
    try {
      await redis.del(key);
    } catch (error) {
      console.error(`Cache DEL error for key ${key}:`, error);
    }
  }
}

// Cache-aware repository
export class CachedConversationRepository extends ConversationRepository {
  private cache = new CacheService();

  async findById(id: string): Promise<Conversation | null> {
    // Try cache first
    const cacheKey = `conversation:${id}`;
    const cached = await this.cache.get<Conversation>(cacheKey);

    if (cached) {
      console.log(`✅ Cache HIT: ${cacheKey}`);
      return cached;
    }

    // Cache miss - fetch from database
    console.log(`❌ Cache MISS: ${cacheKey}`);
    const conversation = await super.findById(id);

    if (conversation) {
      await this.cache.set(cacheKey, conversation, 3600); // 1 hour TTL
    }

    return conversation;
  }
```

```
  async addMessage(conversationId: string, message: Omit<Message, 'id'>): Promi
    // Write to database
    const msg = await super.addMessage(conversationId, message);

    // Invalidate conversation cache
    await this.cache.del(`conversation:${conversationId}`);
    await this.cache.del(`conversation:${conversationId}:latest`);

    return msg;
  }
}
```

Deliverables:

- Redis cluster operational
- Cache hit ratio >70% for conversation reads
- Cache invalidation working correctly
- Monitoring dashboard showing cache metrics

Owner: Backend Developer
Dependencies: Database access layer complete

Week 9-10: S3 Artifact Storage

Tasks:

- [ ] Create S3 bucket with versioning
- [ ] Configure CloudFront distribution
- [ ] Implement S3StorageService

- [ ] Migrate artifact storage from in-memory to S3
- [ ] Add signed URL generation for downloads

Implementation:

```typescript
```

```
// src/lib/storage/s3.service.ts
import { S3Client, PutObjectCommand, GetObjectCommand } from '@aws-sdk/client-s
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

export class S3StorageService {
  private s3Client: S3Client;
  private bucketName = 'igpt-artifacts-prod';
  private cloudFrontDomain = 'd1234abcd.cloudfront.net';

  constructor() {
    this.s3Client = new S3Client({
      region: 'us-east-1',
      credentials: {
        accessKeyId: process.env.AWS_ACCESS_KEY_ID!,
        secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY!
      }
    });
  }

  async uploadArtifact(
    workspaceId: string,
    conversationId: string,
    artifactId: string,
    version: number,
    content: string,
    language: string
  ): Promise<string> {
    const key = `artifacts/${workspaceId}/${conversationId}/${artifactId}/v${ve

    await this.s3Client.send(new PutObjectCommand({
      Bucket: this.bucketName,
      Key: key,
```

```
    Body: content,
    ContentType: this.getContentType(language),
    Metadata: {
      artifactId,
      version: version.toString(),
      language,
      conversationId
    }
  }));

  return `s3://${this.bucketName}/${key}`;
}

// Generate CloudFront signed URL (expires in 1 hour)
async getArtifactUrl(storagePath: string): Promise<string> {
  const key = storagePath.replace(`s3://${this.bucketName}/`, '');

  // For public CDN access, return CloudFront URL
  // For private access, use signed S3 URL
  return `https://${this.cloudFrontDomain}/${key}`;
}

private getExtension(language: string): string {
  const extensions: Record<string, string> = {
    'python': 'py',
    'javascript': 'js',
    'typescript': 'ts',
    'java': 'java',
    'html': 'html',
    'css': 'css',
    'json': 'json',
    'markdown': 'md'
```

```
    };
    return extensions[language.toLowerCase()] || 'txt';
  }

  private getContentType(language: string): string {
    const types: Record<string, string> = {
      'python': 'text/x-python',
      'javascript': 'application/javascript',
      'typescript': 'application/typescript',
      'java': 'text/x-java',
      'html': 'text/html',
      'css': 'text/css',
      'json': 'application/json',
      'markdown': 'text/markdown'
    };
    return types[language.toLowerCase()] || 'text/plain';
  }
}

// Update Artifact Repository
export class ArtifactRepository {
  private storage = new S3StorageService();

  async createArtifact(artifact: CreateArtifactInput): Promise<Artifact> {
    // 1. Upload to S3
    const storagePath = await this.storage.uploadArtifact(
      artifact.workspaceId,
      artifact.conversationId,
      artifact.id,
      1, // Initial version
      artifact.content,
      artifact.language
```

```
  );

  // 2. Create database record
  return await prisma.artifact.create({
    data: {
      id: artifact.id,
      conversationId: artifact.conversationId,
      userId: artifact.userId,
      artifactType: artifact.type,
      title: artifact.title,
      language: artifact.language,
      currentVersion: 1,
      storagePath
    }
  });
}

async updateArtifact(artifactId: string, content: string): Promise<ArtifactVe
  // 1. Get current artifact
  const artifact = await prisma.artifact.findUnique({
    where: { id: artifactId }
  });

  if (!artifact) throw new Error('Artifact not found');

  const newVersion = artifact.currentVersion + 1;

  // 2. Upload new version to S3
  const storagePath = await this.storage.uploadArtifact(
    artifact.workspaceId,
    artifact.conversationId,
    artifactId,
```

```
      newVersion,
      content,
      artifact.language
    );

    // 3. Create version record + update current version (transactional)
    return await prisma.$transaction(async (tx) => {
      const version = await tx.artifactVersion.create({
        data: {
          artifactId,
          version: newVersion,
          content: content.length < 5000 ? content : null, // Store small artif
          sizeBytes: Buffer.byteLength(content, 'utf-8'),
          storagePath
        }
      });

      await tx.artifact.update({
        where: { id: artifactId },
        data: {
          currentVersion: newVersion,
          updatedAt: new Date()
        }
      });

      return version;
    });
  }
}
```

Deliverables:

- S3 bucket configured with lifecycle policies
- CloudFront distribution serving artifacts
- All artifact uploads/downloads using S3
- Artifact versioning working correctly

Owner: Backend Developer
Dependencies: Database schema includes artifact tables

Phase 3: Advanced Features (Weeks 11-14)

Week 11-12: Vector Search

Tasks:

- [ ] Set up Pinecone index
- [ ] Implement VectorSearchService
- [ ] Add background job for embedding generation
- [ ] Create search API endpoint
- [ ] Add search UI component

Code Example:

```typescript
```

```typescript
// src/lib/search/vector-search.service.ts
import { Pinecone } from '@pinecone-database/pinecone';
import OpenAI from 'openai';

export class VectorSearchService {
  private pinecone: Pinecone;
  private openai: OpenAI;
  private indexName = 'igpt-messages-prod';

  constructor() {
    this.pinecone = new Pinecone({
      apiKey: process.env.PINECONE_API_KEY!,
      environment: 'us-east-1-aws'
    });

    this.openai = new OpenAI({
      apiKey: process.env.OPENAI_API_KEY!
    });
  }

  async embedText(text: string): Promise<number[]> {
    const response = await this.openai.embeddings.create({
      model: 'text-embedding-ada-002',
      input: text.substring(0, 8000) // OpenAI limit
    });

    return response.data[0].embedding;
  }

  async indexMessage(message: Message): Promise<void> {
    try {
      const embedding = await this.embedText(message.content);
```

```
    const index = this.pinecone.index(this.indexName);
    await index.upsert([{
      id: message.id,
      values: embedding,
      metadata: {
        conversationId: message.conversationId,
        userId: message.userId,
        role: message.role,
        content: message.content.substring(0, 1000),
        createdAt: message.createdAt.toISOString()
      }
    }]);

    // Update database status
    await prisma.message.update({
      where: { id: message.id },
      data: {
        embeddingStatus: 'indexed',
        embeddingIndexedAt: new Date()
      }
    });

    console.log(`✅ Indexed message ${message.id}`);
  } catch (error) {
    console.error(`❌ Failed to index message ${message.id}:`, error);

    await prisma.message.update({
      where: { id: message.id },
      data: { embeddingStatus: 'failed' }
    });
  }
```

```
}

async searchConversations(
  query: string,
  userId: string,
  limit: number = 10
): Promise<SearchResult[]> {
  const queryEmbedding = await this.embedText(query);

  const index = this.pinecone.index(this.indexName);
  const results = await index.query({
    vector: queryEmbedding,
    topK: limit * 2, // Fetch more, then filter
    filter: { userId: { $eq: userId } },
    includeMetadata: true
  });

  // Group by conversation, deduplicate
  const conversationMap = new Map<string, SearchResult>();

  for (const match of results.matches) {
    const convId = match.metadata.conversationId as string;

    if (!conversationMap.has(convId) || match.score > conversationMap.get(con
      conversationMap.set(convId, {
        conversationId: convId,
        messageId: match.id,
        snippet: match.metadata.content as string,
        score: match.score,
        createdAt: new Date(match.metadata.createdAt as string)
      });
    }
```

```
    }

    return Array.from(conversationMap.values())
      .sort((a, b) => b.score - a.score)
      .slice(0, limit);
  }
}

// Background job (runs every 5 minutes)
async function processEmbeddingQueue() {
  const pendingMessages = await prisma.message.findMany({
    where: { embeddingStatus: 'pending' },
    take: 100,
    orderBy: { createdAt: 'desc' }
  });

  const vectorSearch = new VectorSearchService();

  for (const message of pendingMessages) {
    await vectorSearch.indexMessage(message);
    await new Promise(resolve => setTimeout(resolve, 100)); // Rate limit OpenA
  }
}

setInterval(processEmbeddingQueue, 5 * 60 * 1000); // Every 5 minutes
```

API Endpoint:

```typescript
```

```
// src/app/api/search/route.ts
import { VectorSearchService } from '@/lib/search/vector-search.service';

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const query = searchParams.get('q');
  const userId = searchParams.get('userId');

  if (!query || !userId) {
    return NextResponse.json({ error: 'Missing query or userId' }, { status: 40
  }

  const vectorSearch = new VectorSearchService();
  const results = await vectorSearch.searchConversations(query, userId);

  // Enrich with conversation metadata
  const enrichedResults = await Promise.all(
    results.map(async (result) => {
      const conversation = await prisma.conversation.findUnique({
        where: { id: result.conversationId },
        select: { title: true, updatedAt: true }
      });

      return {
        ...result,
        conversationTitle: conversation?.title,
        conversationUpdatedAt: conversation?.updatedAt
      };
    })
  );
```

```
  return NextResponse.json(enrichedResults);
}
```

Deliverables:

- Pinecone index operational
- Background embedding job running
- Search API endpoint working
- Search UI integrated into chat interface

Owner: Backend Developer + Frontend Developer
Dependencies: Database and S3 storage complete

Week 13-14: Message Queue (Kafka)

Tasks:

- [ ] Provision AWS MSK (Managed Kafka)
- [ ] Implement Kafka producer/consumer
- [ ] Migrate message creation to async queue
- [ ] Add dead letter queue handling
- [ ] Monitor queue lag and throughput

Producer (API Route):

```typescript
```

```typescript
// src/lib/queue/kafka.producer.ts
import { Kafka, Producer } from 'kafkajs';

export class KafkaProducerService {
  private kafka: Kafka;
  private producer: Producer;

  constructor() {
    this.kafka = new Kafka({
      clientId: 'igpt-api',
      brokers: [
        'b-1.igpt-kafka.kafka.us-east-1.amazonaws.com:9092',
        'b-2.igpt-kafka.kafka.us-east-1.amazonaws.com:9092'
      ],
      ssl: true,
      sasl: {
        mechanism: 'scram-sha-256',
        username: process.env.KAFKA_USERNAME!,
        password: process.env.KAFKA_PASSWORD!
      }
    });

    this.producer = this.kafka.producer({
      allowAutoTopicCreation: false,
      transactionTimeout: 30000
    });
  }

  async connect(): Promise<void> {
    await this.producer.connect();
    console.log('✅ Kafka producer connected');
  }
```

```
  async publishMessage(message: Message): Promise<void> {
    await this.producer.send({
      topic: 'messages.created',
      messages: [{
        key: message.conversationId,
        value: JSON.stringify(message),
        partition: this.getPartition(message.userId)
      }]
    });
  }

  private getPartition(userId: string): number {
    // Consistent hash to partition (ensures ordering per user)
    const hash = userId.split('').reduce((acc, char) => acc + char.charCodeAt(0
    return hash % 10; // 10 partitions
  }
}
```

Consumer (Worker Process):

```typescript
```

```typescript
// src/workers/message-writer.worker.ts
import { Kafka, Consumer } from 'kafkajs';
import { MessageRepository } from '@/lib/db/repositories/message.repository';
import { CacheService } from '@/lib/cache/redis';
import { VectorSearchService } from '@/lib/search/vector-search.service';

async function startMessageWriter() {
  const kafka = new Kafka({
    clientId: 'igpt-message-writer',
    brokers: [
      'b-1.igpt-kafka.kafka.us-east-1.amazonaws.com:9092',
      'b-2.igpt-kafka.kafka.us-east-1.amazonaws.com:9092'
    ],
    ssl: true,
    sasl: {
      mechanism: 'scram-sha-256',
      username: process.env.KAFKA_USERNAME!,
      password: process.env.KAFKA_PASSWORD!
    }
  });

  const consumer = kafka.consumer({
    groupId: 'message-writers',
    sessionTimeout: 30000,
    heartbeatInterval: 3000
  });

  await consumer.connect();
  await consumer.subscribe({ topic: 'messages.created', fromBeginning: false })

  const messageRepo = new MessageRepository();
  const cache = new CacheService();
```

```
const vectorSearch = new VectorSearchService();

console.log('✅ Message writer consumer started');

await consumer.run({
  eachMessage: async ({ topic, partition, message }) => {
    const startTime = Date.now();
    const msg = JSON.parse(message.value!.toString()) as Message;

    try {
      console.log(`📝 Processing message ${msg.id}...`);

      // 1. Write to PostgreSQL (with retry)
      await messageRepo.create(msg);
      console.log(`✅ [${Date.now() - startTime}ms] DB write complete`);

      // 2. Invalidate cache
      await cache.del(`conversation:${msg.conversationId}:latest`);
      console.log(`✅ [${Date.now() - startTime}ms] Cache invalidated`);

      // 3. Queue embedding job (async, non-blocking)
      await vectorSearch.indexMessage(msg);
      console.log(`✅ [${Date.now() - startTime}ms] Embedding queued`);

      console.log(`✅ [${Date.now() - startTime}ms] Message ${msg.id} process
    } catch (error) {
      console.error(`❌ Failed to process message ${msg.id}:`, error);

      // Send to dead letter queue
      await kafka.producer().send({
        topic: 'messages.dlq',
        messages: [{
```

```
            key: msg.id,
            value: JSON.stringify({
              message: msg,
              error: String(error),
              timestamp: new Date().toISOString()
            })
          }]
        });
      }
    }
  });
}

// Start worker
startMessageWriter().catch(console.error);
```

Updated API Route:

```typescript
```

```typescript
// src/app/api/conversations/[id]/messages/route.ts
import { KafkaProducerService } from '@/lib/queue/kafka.producer';

const kafkaProducer = new KafkaProducerService();
await kafkaProducer.connect(); // Connect once on server start

export async function POST(request: Request) {
  const { message, conversationId, userId } = await request.json();

  // Create message object
  const msg: Message = {
    id: generateId(),
    conversationId,
    userId,
    role: 'user',
    content: message,
    createdAt: new Date(),
    embeddingStatus: 'pending'
  };

  // Publish to Kafka (returns immediately)
  await kafkaProducer.publishMessage(msg);

  // Return 202 Accepted (processing in background)
  return NextResponse.json(
    {
      messageId: msg.id,
      status: 'accepted',
      message: 'Message queued for processing'
    },
    { status: 202 }
```

```
  );
}
```

Deliverables:

- Kafka cluster operational
- Producer publishing messages
- Consumer writing to database
- Dead letter queue handling errors
- Monitoring dashboard showing queue lag

Owner: Backend Engineer
Dependencies: Database and caching layers complete

Phase 4: Observability & Monitoring (Weeks 15-16)

Week 15: Logging & Metrics

Tasks:

- [ ] Set up CloudWatch Logs
- [ ] Implement structured logging
- [ ] Add Prometheus metrics exporters
- [ ] Create Grafana dashboards
- [ ] Set up alerting rules

Structured Logging:

```typescript
```

```typescript
// src/lib/logging/logger.ts
import winston from 'winston';
import CloudWatchTransport from 'winston-cloudwatch';

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: {
    service: 'igpt-api',
    environment: process.env.NODE_ENV
  },
  transports: [
    // Console (development)
    new winston.transports.Console({
      format: winston.format.simple()
    }),

    // CloudWatch (production)
    new CloudWatchTransport({
      logGroupName: '/aws/igpt/api',
      logStreamName: `${process.env.NODE_ENV}-${Date.now()}`,
      awsRegion: 'us-east-1'
    })
  ]
});

export default logger;
```

```
// Usage
logger.info('Message created', {
  messageId: msg.id,
  conversationId: msg.conversationId,
  userId: msg.userId,
  tokensUsed: msg.tokensUsed,
  model: msg.model
});
```

Prometheus Metrics:

```typescript
```

```
// src/lib/metrics/prometheus.ts
import { Counter, Histogram, Gauge, register } from 'prom-client';

export const metrics = {
  // HTTP requests
  httpRequestsTotal: new Counter({
    name: 'http_requests_total',
    help: 'Total HTTP requests',
    labelNames: ['method', 'route', 'status']
  }),

  httpRequestDuration: new Histogram({
    name: 'http_request_duration_seconds',
    help: 'HTTP request duration',
    labelNames: ['method', 'route'],
    buckets: [0.1, 0.5, 1, 2, 5]
  }),

  // LLM requests
  llmRequestsTotal: new Counter({
    name: 'llm_requests_total',
    help: 'Total LLM requests',
    labelNames: ['provider', 'model', 'status']
  }),

  llmTokensUsed: new Counter({
    name: 'llm_tokens_used_total',
    help: 'Total tokens used',
    labelNames: ['provider', 'model']
  }),

  // Database
```

```
  dbQueriesTotal: new Counter({
    name: 'db_queries_total',
    help: 'Total database queries',
    labelNames: ['operation', 'table']
  }),

  dbQueryDuration: new Histogram({
    name: 'db_query_duration_seconds',
    help: 'Database query duration',
    labelNames: ['operation', 'table'],
    buckets: [0.01, 0.05, 0.1, 0.5, 1]
  }),

  // Cache
  cacheHitsTotal: new Counter({
    name: 'cache_hits_total',
    help: 'Total cache hits',
    labelNames: ['cache_key_pattern']
  }),

  cacheMissesTotal: new Counter({
    name: 'cache_misses_total',
    help: 'Total cache misses',
    labelNames: ['cache_key_pattern']
  }),

  // Active connections
  activeWebsocketConnections: new Gauge({
    name: 'active_websocket_connections',
    help: 'Number of active WebSocket connections'
  })
};
```

```
// Metrics endpoint
export async function GET() {
  return new Response(await register.metrics(), {
    headers: { 'Content-Type': register.contentType }
  });
}
```

Grafana Dashboard Config:

```yaml
```

```yaml
Dashboard: IGPT Production Monitoring

Panels:
  - Row 1: System Health
    - HTTP Request Rate (requests/sec)
    - HTTP Error Rate (%)
    - API Response Time (p50, p95, p99)
    - Active Users (gauge)

  - Row 2: LLM Metrics
    - LLM Requests by Provider (stacked area)
    - Token Usage by Model (bar chart)
    - LLM Error Rate (%)
    - Average Response Time (line)

  - Row 3: Database
    - Query Rate (queries/sec)
    - Query Duration (heatmap)
    - Connection Pool Usage (%)
    - Slow Queries (>1s)

  - Row 4: Cache
    - Cache Hit Rate (%)
    - Cache Memory Usage (GB)
    - Cache Evictions (rate)

  - Row 5: Infrastructure
    - CPU Usage (%)
    - Memory Usage (%)
    - Disk I/O (MB/s)
    - Network Traffic (MB/s)
```

```
Alerts:
  - HTTP Error Rate > 5% for 5 minutes
  - API Response Time p99 > 5 seconds
  - Database Connection Pool > 90%
  - LLM Error Rate > 10%
  - Cache Hit Rate < 60%
```

Deliverables:

- Structured logging to CloudWatch
- Prometheus metrics exposed at `/metrics`
- Grafana dashboard deployed
- Alerting rules configured
- On-call rotation set up (PagerDuty integration)

Owner: DevOps Engineer
Dependencies: All infrastructure deployed

Week 16: Analytics & Compliance

Tasks:

- [ ] Implement audit logging
- [ ] Create usage analytics pipeline
- [ ] Set up compliance reports
- [ ] Document data retention policies
- [ ] Conduct security audit

Audit Logging:

```typescript
```

```
// src/lib/audit/audit-logger.ts
export class AuditLogger {
  async log(event: AuditEvent): Promise<void> {
    await prisma.auditLog.create({
      data: {
        userId: event.userId,
        action: event.action,
        resourceType: event.resourceType,
        resourceId: event.resourceId,
        ipAddress: event.ipAddress,
        userAgent: event.userAgent,
        metadata: event.metadata
      }
    });
  }
}

// Usage in API routes
const auditLogger = new AuditLogger();

export async function DELETE(request: Request, { params }: { params: { id: stri
  const conversationId = params.id;
  const userId = request.headers.get('x-user-id');

  // Delete conversation
  await conversationRepo.delete(conversationId);

  // Audit log
  await auditLogger.log({
    userId,
    action: 'conversation.deleted',
    resourceType: 'conversation',
```

```
    resourceId: conversationId,
    ipAddress: request.headers.get('x-forwarded-for'),
    userAgent: request.headers.get('user-agent'),
    metadata: { deletedAt: new Date().toISOString() }
  });

  return NextResponse.json({ success: true });
}
```

Usage Analytics Pipeline:

```typescript
```

```
// src/lib/analytics/usage-tracker.ts
export class UsageTracker {
  async trackMessageCreated(message: Message, tokensUsed: number, costUsd: numb
    await prisma.usageLog.create({
      data: {
        userId: message.userId,
        conversationId: message.conversationId,
        eventType: 'message_sent',
        tokensUsed,
        costUsd,
        model: message.model,
        provider: message.provider
      }
    });
  }

  async getMonthlyUsage(userId: string, month: string): Promise<UsageSummary> {
    const result = await prisma.usageLog.aggregate({
      where: {
        userId,
        createdAt: {
          gte: new Date(`${month}-01`),
          lt: new Date(`${month}-31`)
        }
      },
      _sum: {
        tokensUsed: true,
        costUsd: true
      },
      _count: true
    });
```

```
    return {
      totalMessages: result._count,
      totalTokens: result._sum.tokensUsed || 0,
      totalCost: result._sum.costUsd || 0
    };
  }
}
```

Compliance Report Generator:

```typescript
```

```
// Generate monthly compliance report
async function generateComplianceReport(month: string) {
  const report = {
    reportDate: new Date().toISOString(),
    period: month,

    // User activity
    activeUsers: await prisma.user.count({
      where: {
        lastActiveAt: {
          gte: new Date(`${month}-01`),
          lt: new Date(`${month}-31`)
        }
      }
    }),

    // Data retention
    conversationsCreated: await prisma.conversation.count({
      where: {
        createdAt: {
          gte: new Date(`${month}-01`),
          lt: new Date(`${month}-31`)
        }
      }
    }),

    conversationsDeleted: await prisma.auditLog.count({
      where: {
        action: 'conversation.deleted',
        createdAt: {
          gte: new Date(`${month}-01`),
          lt: new Date(`${month}-31`)
```

```
      }
    }
  }),

  // Audit trail
  auditLogEntries: await prisma.auditLog.count({
    where: {
      createdAt: {
        gte: new Date(`${month}-01`),
        lt: new Date(`${month}-31`)
      }
    }
  }),

  // Security events
  failedLoginAttempts: await prisma.auditLog.count({
    where: {
      action: 'auth.login_failed',
      createdAt: {
        gte: new Date(`${month}-01`),
        lt: new Date(`${month}-31`)
      }
    }
  })
};

// Export to PDF
const pdf = await generatePDF(report);

// Upload to S3 (compliance bucket)
await s3.upload({
  Bucket: 'igpt-compliance-reports',
```

```
    Key: `reports/${month}-compliance-report.pdf`,
    Body: pdf
  });

  console.log(`✅ Compliance report generated for ${month}`);
}

// Run on 1st of every month
cron.schedule('0 0 1 * *', () => {
  const lastMonth = new Date();
  lastMonth.setMonth(lastMonth.getMonth() - 1);
  const month = lastMonth.toISOString().substring(0, 7); // YYYY-MM

  generateComplianceReport(month);
});
```

Deliverables:

- Audit logging operational for all critical actions
- Usage analytics dashboard
- Automated compliance reports
- Data retention policy documented
- Security audit completed

Owner: Security Engineer + Compliance Officer
Dependencies: All systems operational

---

5. Technical Specifications

5.1 Database Schema Summary

Tables: 11 core tables

- `users`, `workspaces`, `workspace_members`
- `conversations`, `messages`
- `artifacts`, `artifact_versions`
- `attachments`
- `usage_logs`, `audit_logs`
- `prompt_templates`

Indexes: 25+ indexes for performance **Sharding:** 8 shards, partitioned by `user_id`
Replication: 3x replication per shard (24 total instances)

5.2 API Endpoints

RESTful API:

```
GET    /api/conversations              - List user's conversations
POST   /api/conversations              - Create new conversation
GET    /api/conversations/:id          - Get conversation details
DELETE /api/conversations/:id          - Delete conversation
PUT    /api/conversations/:id          - Update conversation (title, status)

POST   /api/conversations/:id/messages - Create message (async via Kafka)
GET    /api/conversations/:id/messages - Get conversation messages

GET    /api/artifacts                  - List artifacts
POST   /api/artifacts                  - Create artifact
GET    /api/artifacts/:id              - Get artifact (specific version)
PUT    /api/artifacts/:id              - Update artifact (creates new
version)
GET    /api/artifacts/:id/versions     - List all versions
GET    /api/artifacts/:id/download     - Get signed URL for download

POST   /api/attachments/upload         - Upload file to S3
GET    /api/attachments/:id            - Get attachment metadata

GET    /api/search                     - Vector search conversations
GET    /api/usage                      - Get usage statistics

GET    /api/health                     - Health check
GET    /api/metrics                    - Prometheus metrics
```

GraphQL API (Future):

```graphql
```

```
type Query {
  conversation(id: ID!): Conversation
  conversations(userId: ID!, limit: Int): [Conversation!]!
  searchMessages(query: String!, userId: ID!): [Message!]!
  artifacts(conversationId: ID!): [Artifact!]!
}

type Mutation {
  createConversation(title: String!): Conversation!
  sendMessage(conversationId: ID!, content: String!): Message!
  createArtifact(input: ArtifactInput!): Artifact!
  updateArtifact(id: ID!, content: String!): ArtifactVersion!
}

type Subscription {
  messageAdded(conversationId: ID!): Message!
  typingIndicator(conversationId: ID!): TypingEvent!
}
```

5.3 Infrastructure Components

Compute:

- Next.js API: 4x t3.large instances (2 vCPU, 8 GB RAM each)
- Kafka Workers: 4x t3.large instances
- Total: 8 application servers

Database:

- PostgreSQL: 8 shards × 3 replicas = 24x db.r5.large (2 vCPU, 16 GB RAM)

- Redis: 3-node cluster, cache.r6g.large (13 GB RAM per node)

Storage:

- S3: 50 TB artifact storage
- CloudFront: Global CDN
- EBS: 2 TB for database logs

Message Queue:

- AWS MSK: 3-broker Kafka cluster (kafka.m5.large)

Vector Database:

- Pinecone: 1x s1.x1 pod (1M vectors)

Monitoring:

- CloudWatch Logs
- Prometheus + Grafana
- DataDog (optional)

---

6. Cost Analysis

6.1 Monthly Infrastructure Costs (100k Daily Users)

Component	Configuration	Monthly Cost
Compute		

Component	Configuration	Monthly Cost
Next.js API Servers	4x t3.large	$480
Kafka Worker Servers	4x t3.large	$480
Database		
PostgreSQL (Aurora)	8 shards × 3 replicas	$15,000
Redis (ElastiCache)	3-node cluster	$450
Storage		
S3 Standard	10 TB (first year)	$230
CloudFront	5 TB transfer/month	$425
Message Queue		
AWS MSK	3-broker cluster	$1,500
Vector Database		
Pinecone	1x s1.x1 pod	$70
Monitoring		
CloudWatch	Logs + Metrics	$200
Grafana Cloud	Optional	$0 (self-hosted)
Total Infrastructure		**$18,835/month**

6.2 Variable Costs (LLM Usage)

Assumptions:

- 100k daily users
- 5 conversations per user per day
- 20 messages per conversation
- 10M messages/day

Token Usage (per message):

- User message: ~200 tokens
- Assistant response: ~500 tokens
- Total: ~700 tokens/message

Daily Token Usage:

- 10M messages × 700 tokens = 7 billion tokens/day

Monthly Cost (Claude 3.5 Sonnet):

- Input: $3/million tokens
- Output: $15/million tokens
- Blended average: ~$10/million tokens

Monthly LLM Cost:

- 7B tokens/day × 30 days = 210B tokens/month
- 210B tokens × $10/million = **$2,100,000/month**

Cost per User:

- $2.1M / 100k users = **$21/user/month** (LLM only)

- Total (infra + LLM): ~$21.20/user/month

Pricing Strategy:

- Free Tier: 50 messages/month (limited to GPT-3.5 or Claude Haiku)
- Pro Tier: $20/month (unlimited messages, Claude Sonnet)
- Enterprise: Custom pricing (volume discounts, dedicated infrastructure)

---

7. Risk Assessment

7.1 Technical Risks

1. Database Scalability

- **Risk:** PostgreSQL shards may not handle 50k writes/sec
- **Mitigation:**
  - Start with 8 shards, scale to 16 if needed
  - Use read replicas for analytics queries
  - Implement aggressive caching (80%+ hit rate)
  - Batch writes via Kafka (reduces DB load by 5x)

2. LLM Provider Downtime

- **Risk:** AWS Bedrock or OpenAI outages block all conversations
- **Mitigation:**
  - Multi-provider fallback (Bedrock → OpenAI → Google)
  - Circuit breaker pattern (auto-failover)

    - Queue requests during outages (up to 1 hour buffer)
    - Show graceful error messages to users

3. Cost Overruns

- **Risk:** LLM costs exceed projections ($2.1M/month)
- **Mitigation:**
    - Token usage limits per user tier
    - Real-time cost tracking dashboard
    - Alert when daily spend exceeds $75k
    - Implement aggressive context window optimization

4. Data Loss

- **Risk:** Database corruption or accidental deletion
- **Mitigation:**
    - Daily full backups + 6-hour incremental
    - Point-in-time recovery (30 days)
    - Multi-region replication (failover to us-west-2)
    - Regular restore drills (monthly)

7.2 Security Risks

1. Data Breach

- **Risk:** Unauthorized access to user conversations
- **Mitigation:**

    - Encryption at rest (AES-256) and in transit (TLS 1.3)
    - Row-level security (users can only access their own data)
    - Regular penetration testing (quarterly)
    - SOC 2 Type II compliance

2. DDoS Attack

- **Risk:** Overwhelming traffic takes down service
- **Mitigation:**
    - AWS Shield Standard (free, basic DDoS protection)
    - AWS WAF (rate limiting, geo-blocking)
    - CloudFront CDN (absorbs traffic spikes)
    - Auto-scaling (scale to 50 instances if needed)

3. Injection Attacks

- **Risk:** SQL injection, prompt injection, XSS
- **Mitigation:**
    - Parameterized queries (Prisma ORM)
    - Input validation at API Gateway
    - Content Security Policy headers
    - Sanitize user input before passing to LLM

7.3 Compliance Risks

1. GDPR Violations

- **Risk:** Improper handling of EU user data
- **Mitigation:**
  - Data residency options (EU-only shards)
  - Right to deletion (hard delete within 30 days)
  - Data export API (JSON format)
  - Consent management system

2. Audit Trail Gaps

- **Risk:** Unable to prove compliance during audit
- **Mitigation:**
  - Comprehensive audit logging (all CRUD operations)
  - Immutable logs (append-only tables)
  - 7-year retention for audit logs
  - Monthly compliance reports

---

8. Next Steps & Action Plan

8.1 Immediate Actions (Week 1)

Priority 1: Database Setup

- [ ] Provision AWS Aurora PostgreSQL cluster
- [ ] Deploy schema to development environment
- [ ] Test connection from Next.js app
- [ ] Document connection strings and credentials

Priority 2: Team Alignment

- [ ] Schedule kick-off meeting with full engineering team
- [ ] Assign ownership for each phase
- [ ] Set up weekly sync meetings (Mondays at 10 AM)
- [ ] Create Slack channel: #stage3-storage-architecture

Priority 3: Infrastructure Preparation

- [ ] Request AWS quota increases (Aurora instances)
- [ ] Set up development/staging environments
- [ ] Configure CI/CD pipelines
- [ ] Create feature flags for gradual rollout

8.2 Success Criteria

Phase 1 (Database Foundation):

- ✅ PostgreSQL cluster operational with 99.9% uptime
- ✅ All API routes using database instead of mock data
- ✅ Load testing passes: 1000 req/sec with <500ms p95 latency
- ✅ Zero data loss during migration

Phase 2 (Caching & Performance):

- ✅ Redis cache hit rate >70% for conversation reads
- ✅ S3 artifact storage operational with CloudFront CDN
- ✅ Artifact versioning working correctly
- ✅ API response time reduced by 50% vs. Phase 1

Phase 3 (Advanced Features):

- ✅ Vector search functional with <2 second response time
- ✅ Background embedding queue processing 1000+ messages/minute
- ✅ Kafka message queue handling 10k messages/sec
- ✅ Zero message loss during queue failures

Phase 4 (Observability):

- ✅ Grafana dashboards showing all key metrics
- ✅ Alerting rules triggering correctly (tested)
- ✅ Audit logging capturing 100% of critical actions
- ✅ Compliance reports generated automatically

8.3 Go-Live Checklist

Pre-Production:

- [ ] All phases complete and tested
- [ ] Security audit passed
- [ ] Performance testing passed (10k concurrent users)
- [ ] Disaster recovery plan documented and tested
- [ ] On-call rotation staffed (24/7 coverage)

Production Deployment:

- [ ] Deploy to production environment (Friday evening)
- [ ] Enable feature flag for 1% of users
- [ ] Monitor for 24 hours

- [ ] Gradually increase to 10%, 50%, 100% over 1 week

Post-Launch:

- [ ] Monitor error rates and latency for 2 weeks
- [ ] Collect user feedback
- [ ] Document lessons learned
- [ ] Plan Stage 4 enhancements

Conclusion

This comprehensive analysis outlines a production-ready storage architecture for IGPT, capable of handling 100,000 daily users generating 10 million messages per day.

Key Highlights:

- **PostgreSQL with sharding** provides reliable, scalable data storage
- **Redis caching** reduces database load by 70%+
- **S3 + CloudFront** delivers artifacts globally with <50ms latency
- **Kafka message queue** ensures reliable async processing
- **Pinecone vector search** enables semantic conversation discovery
- **Comprehensive observability** with logging, metrics, and alerting

Timeline: 16 weeks from start to production deployment
Total Investment: ~$20k/month infrastructure + $2.1M/month LLM costs
ROI: Enterprise-grade reliability and scalability to support business growth

Next Step: Schedule kick-off meeting and begin Phase 1 (Database Foundation).

Document Version: 1.0
Last Updated: March 24, 2026
Prepared By: IGPT Engineering Team
Review Status: Pending approval from CTO and Product Lead